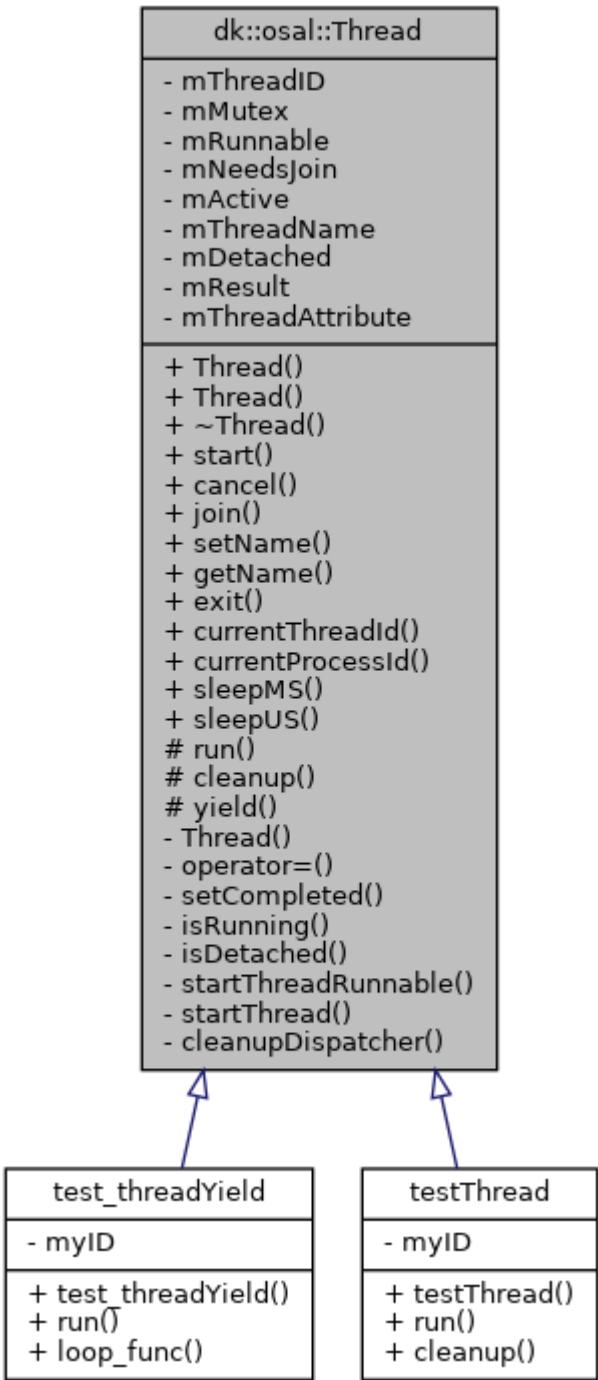


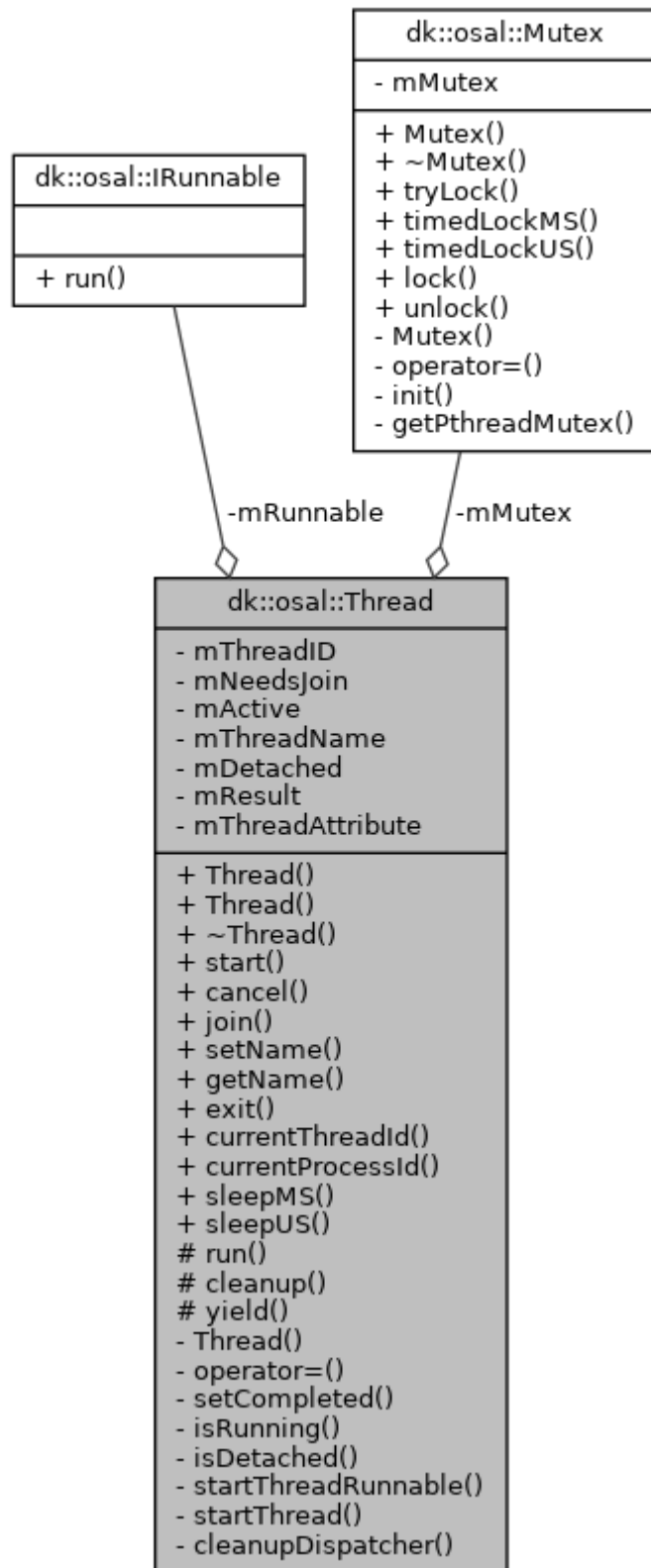
dk::osal::Thread Class Reference

```
#include <DKOsalThread.h>
```

Inheritance diagram for dk::osal::Thread:



Collaboration diagram for dk::osal::Thread:



Public Member Functions

Thread (**IRunnable** *runnable, bool **isDetached**=false)

Construct a new **Thread** with a **IRunnable** object. [More...](#)

Thread (bool **isDetached**=false)

construct a new thread object [More...](#)

~Thread ()

void **start** (**ThreadConfigID** id)

Start Executing the thread. If a **IRunnable** object was supplied in the constructor, its **run()** method is invoked; otherwise the thread object's **run()** method is called. If the thread is already running the call has no effect. [More...](#)

void **cancel** ()

stops this thread. [More...](#)

void * **join** ()

Wait for the thread to terminate. [More...](#)

void **setName** (const std::string &name)

Set the name of this thread. [More...](#)

std::string **getName** ()

Get the name of this thread. [More...](#)

Static Public Member Functions

static void **exit** ()

Terminate the calling thread. If called from the main thread, this function will block until all other threads are finished. [More...](#)

static uint32_t **currentThreadId** ()

Get the **Thread** id (TID) from the calling thread. [More...](#)

static uint32_t **currentProcessId** ()

Get the process id (PID) from the current process. [More...](#)

static void **sleepMS** (uint32_t msec)

Blocks the calling thread for the specified time. [More...](#)

static void **sleepUS** (uint32_t usec)

Blocks the calling thread for the specified time. [More...](#)

Protected Member Functions

virtual void * **run** ()

Main function. **Thread** execution begins in this function. The default implementation does nothing. [More...](#)

virtual void **cleanup** ()

Cleanup function. This function will be executed by the thread just prior to termination. The default implementation is a no-operation. [More...](#)

void **yield** ()

Yield the CPU to another thread with same priority. [More...](#)

Private Member Functions

Thread (const **Thread** &)

Disable copy-constructor and assignment operator to avoid copy operation and assignment operations if someone tries a copy/assignment will get a compiler error. [More...](#)

const **Thread** & **operator=** (const **Thread** &)

void **setCompleted** ()

Called when run completes. [More...](#)

bool **isRunning** () const

Test if the thread is currently running. [More...](#)

bool **isDetached** () const

if the thread is detached [More...](#)

Static Private Member Functions

static void * **startThreadRunnable** (void *parg)

method for threads created from **IRunnable** objects [More...](#)

static void * **startThread** (void *parg)

method for threads created by inheriting from Class **Thread** [More...](#)

static void **cleanupDispatcher** (void *parg)

method for threads created by inheriting from Class **Thread** [More...](#)

Private Attributes

pthread_t **mThreadID**

Mutex **mMutex**

IRunnable * **mRunnable**

bool **mNeedsJoin**

bool **mActive**

std::string **mThreadName**

bool **mDetached**

void * **mResult** = NULL

pthread_attr_t **mThreadAttribute**

Constructor & Destructor Documentation

◆ **Thread**() [1/3]

```
dk::osal::Thread::Thread ( IRunnable * runnable,
                           bool      isDetached = false
                           )
```

Construct a new **Thread** with a **IRunnable** object.

Parameters

runnable The **IRunnable** object whose **run()** method will be executed in this thread.

isDetached A flag indicating whether the thread will be created in a detached state.

requirement id 751552

Here is the call graph for this function:



◆ Thread() [2/3]

```
dk::osal::Thread::Thread ( bool isDetached = false )
```

construct a new thread object

Destructor for the **Thread** class.

Parameters

isDetached A flag indicating whether the thread will be created in a detached state.

requirement id 751552

◆ ~Thread()

```
dk::osal::Thread::~Thread ( )
```

◆ Thread() [3/3]

dk::osal::Thread::Thread (const **Thread** &)

private

Disable copy-constructor and assignment operator to avoid copy operation and assignment operations if someone tries a copy/assignment will get a compiler error.

requirement id 751555

Member Function Documentation

◆ cancel()

void dk::osal::Thread::cancel ()

stops this thread.

requirement id 751555

◆ cleanup()

void dk::osal::Thread::cleanup ()

protected

virtual

Cleanup function. This function will be executed by the thread just prior to termination. The default implementation is a no-operation.

requirement id 751555

Reimplemented in **testThread**.

◆ cleanupDispatcher()

static void dk::osal::Thread::cleanupDispatcher (void * **parg**)

static

private

method for threads created by inheriting from Class **Thread**

requirement id 751555

◆ currentProcessId()

```
static uint32_t dk::osal::Thread::currentProcessId ( )
```

static

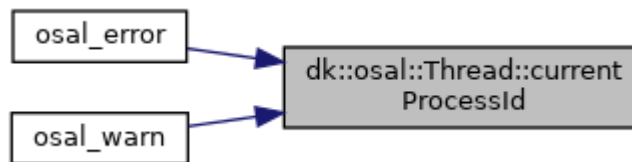
Get the process id (PID) from the current process.

Returns

the process id.

requirement id 751555

Here is the caller graph for this function:



◆ currentThreadId()

```
uint32_t dk::osal::Thread::currentThreadId ( )
```

static

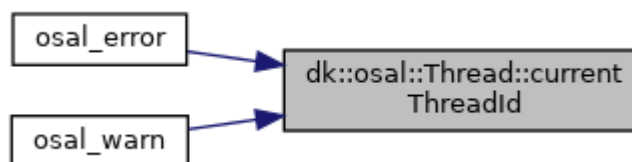
Get the **Thread** id (TID) from the calling thread.

Returns

the thread id.

requirement id 751555

Here is the caller graph for this function:



◆ exit()

```
static void dk::osal::Thread::exit ( )
```

static

Terminate the calling thread. If called from the main thread, this function will block until all other threads are finished.

requirement id 751555

◆ getName()

```
std::string dk::osal::Thread::getName ( )
```

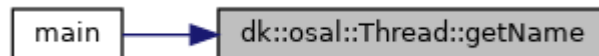
Get the name of this thread.

Returns

name of the thread.

requirement id 751555

Here is the caller graph for this function:



◆ isDetached()

```
bool dk::osal::Thread::isDetached ( ) const
```

private

if the thread is detached

Returns

true if in detached state, **false** otherwise

requirement id 751555

◆ isRunning()

```
bool dk::osal::Thread::isRunning ( ) const
```

private

Test if the thread is currently running.

Returns

true thread is running, **false** not started or finished

requirement id 751555

◆ join()

```
void * dk::osal::Thread::join ( )
```

Wait for the thread to terminate.

requirement id 751555

◆ operator=()

```
const Thread& dk::osal::Thread::operator= ( const Thread & )
```

private

◆ run()

```
void * dk::osal::Thread::run ( )
```

protected

virtual

Main function. **Thread** execution begins in this function. The default implementation does nothing.

requirement id 751555

Reimplemented in **testThread**, and **test_threadYield**.

◆ setCompleted()

```
void dk::osal::Thread::setCompleted ( )
```

private

Called when run completes.

requirement id 751555

◆ setName()

```
void dk::osal::Thread::setName ( const std::string & name )
```

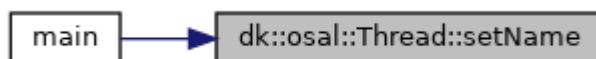
Set the name of this thread.

Parameters

name thread name to be set

requirement id 751555

Here is the caller graph for this function:



◆ sleepMS()

```
static void dk::osal::Thread::sleepMS ( uint32_t msec )
```

static

Blocks the calling thread for the specified time.

Parameters

msec the time to sleep in milliseconds

requirement id 751555

◆ sleepUS()

```
static void dk::osal::Thread::sleepUS ( uint32_t usec )
```

static

Blocks the calling thread for the specified time.

Parameters

nusec the time to sleep in microseconds.

requirement id 751555

◆ start()

```
void dk::osal::Thread::start ( ThreadConfigID id )
```

Start Executing the thread. If a **IRunnable** object was supplied in the constructor, its **run()** method is invoked; otherwise the thread object's **run()** method is called. If the thread is already running the call has no effect.

Parameters

id ID for thread parameter configuration

requirement id 751555

◆ startThread()

```
static void * dk::osal::Thread::startThread ( void * parg )
```

static

private

method for threads created by inheriting from Class **Thread**

requirement id 751555

◆ startThreadRunnable()

```
static void * dk::osal::Thread::startThreadRunnable ( void * parg )
```

static

private

method for threads created from **IRunnable** objects

requirement id 751555

◆ yield()

```
void dk::osal::Thread::yield ( )
```

protected

Yield the CPU to another thread with same priority.

requirement id 751555

Member Data Documentation

◆ mActive

bool dk::osal::Thread::mActive

private

◆ mDetached

bool dk::osal::Thread::mDetached

private

◆ mMutex

Mutex dk::osal::Thread::mMutex

mutable

private

◆ mNeedsJoin

bool dk::osal::Thread::mNeedsJoin

private

◆ mResult

void* dk::osal::Thread::mResult = NULL

private

◆ mRunnable

IRunnable* dk::osal::Thread::mRunnable

private

◆ mThreadAttribute

pthread_attr_t dk::osal::Thread::mThreadAttribute

private

◆ mThreadID

pthread_t dk::osal::Thread::mThreadID

private

◆ mThreadName

std::string dk::osal::Thread::mThreadName

private

The documentation for this class was generated from the following files:

- /home/damudhar/29_8_audi/cluster-platform/dijkstra/abstraction/osal/src/posix/[DKOsaiThread.h](#)
- /home/damudhar/29_8_audi/cluster-platform/dijkstra/abstraction/osal/src/posix/[DKOsaiThread.cpp](#)

Generated by  1.8.17